

Domande per l'Orale LP

Modulo I/II/III

Contents

1	Modulo 1 – Gorrieri	2
1.1	Capitolo 1 – Macchine astratte, interpreti, compilatori	2
1.2	Capitolo 2 – Descrivere un linguaggio di programmazione	5
1.3	Capitolo 3 – Analisi lessicale, linguaggi regolari	11
1.4	Capitolo 4 – Analisi sintattica, linguaggi liberi	22
1.5	Capitolo 5 – Fondamenti	33

1 Modulo 1 – Gorrieri

1.1 Capitolo 1 – Macchine astratte, interpreti, compilatori

Domanda 1

Che cosa è una macchina astratta? E in cosa si differenzia da una macchina fisica?

E' un insieme di algoritmi e strutture dati che dato un programma scritto in un linguaggio relativo alla macchina astratta, quest'ultima lo esegue.

Una macchina fisica e' un insieme di circuiti logici composti da circuiti elettrici che allo stesso modo della macchina astratta dato un linguaggio specifico (conforme all'IS), la macchina fisica lo esegue.

Domanda 2

Che cos'è un interprete? In cosa consiste il ciclo fetch-decode-execute?

Un interprete e' un programma che viene eseguito su un calcolatore che serve per eseguire in realtime un programma, senza prima doverlo compilare. Serve per poter eseguire un linguaggio $\mathcal{L}_A$ su un calcolatore che nativamente esegue il linguaggio $\mathcal{L}$. L'interprete esegue il codice leggendo una istruzione alla volta.

FDE e' lo schema della CPU nel quale prende, decodifica ed esegue una istruzione. Questo e' fortemente collegato alla pipeline della CPU.

Domanda 3

Cos'è il linguaggio macchina?

E' linguaggio relativo all'interprete $I_{\mathcal{L}}$ per la macchina $M_{\mathcal{L}}$.

Domanda 4

Possono esistere macchine diverse con lo stesso linguaggio macchina?

Certamente, infatti possiamo avere macchine astratte o fisiche che implementano o adattano ISA a seconda del linguaggio.

Domanda 5

In quali modi è possibile implementare una macchina astratta? Elencare vantaggi e svantaggi delle varie tecniche.

In 3 modi:

- 1. A livello software: piu' lento ma il piu' flessibile*
- 2. A livello firmware: abbastanza veloce, abbastanza flessibile*
- 3. A livello hardware: piu' veloce ma meno flessibile*

Questo perche' ogni aumento di livello porta un overhead maggiore, quindi abbiamo un tradeoff fra performance e velocita' di implementazione da parte del programmatore.

Domanda 6

Che cos'e un compilatore?

Un compilatore e' un programma che traduce programma scritti in $\mathcal{L}$ in altri linguaggi in $\mathcal{L}_0$:

$$C_{\mathcal{L}, \mathcal{L}_0} : P^{\mathcal{L}} \rightarrow P^{\mathcal{L}_0}$$

Domanda 7

Relativamente alla tecnica d'implementazione software, descrivere la tecnica d'implementazione interpretativa pura e quella compilativa pura.

Il compilatore e' un traduttore da un linguaggio $\mathcal{L}$ a un altro linguaggio $\mathcal{L}_0$, questo traduce da inizio a fine e ritorna un nuovo programma in $\mathcal{L}_0$. Mentre un interprete e' una macchina astratta dove esegue riga per riga alla volta una operazione.

Domanda 8

Quando un interprete si puo dire corretto? Quando un compilatore si puo dire corretto?

Un interprete o compilatore si puo' dire corretto quando l'output restituito dal programma/esecuzione combacia di relazione dominio $\Rightarrow$ codominio con quello nativo.

Domanda 9

Confrontare l'implementazione di una macchina astratta su una macchina ospite per mezzo di un interprete o di un compilatore.

Data una macchina fisica M con linguaggio $\mathcal{L}$ possiamo eseguire programmi scritti in $\mathcal{L}_0$ grazie a un interprete o compilatore t.c.:

$$I_{\mathcal{L}}^{\mathcal{L}_0}$$

L'interpretazione esegue ogni istruzione di $\mathcal{L}_0$, istruzione per istruzione, sulla macchina astratta che gira in M . Mentre il compilatore traduce a priori e poi se si vuole si esegue il nuovo programma tradotto da $\mathcal{L}_0$ in linguaggio nativo $\mathcal{L}$.

Domanda 10

Come vengono implementate nella realta le macchine astratte? Che cos'e la macchina intermedia?

In una macchina fisica per implementare il linguaggio $\mathcal{L}_0$ possiamo tradurlo prima in un IR/Bytecode $\mathcal{L}_1$ che verra' eseguito su una macchina astratta (intermediaria) fra $\mathcal{L}$ e $\mathcal{L}_0$:

$$\mathcal{L}_0 \rightarrow \mathcal{L}_1 \rightarrow \mathcal{L}$$

Domanda 11

Quando si dice che una implementazione e di tipo interpretativo e quando di tipo compilativo? Fare esempi di linguaggi la cui implementazione e di un tipo o dell'altro.

E' interpretativo quando utilizziamo un interprete, mentre e' compilativo quando utilizziamo un compilatore. Per esempio python e' interpretativo, mentre C e' compilativo.

Domanda 12

Cos'e la Just-in-Time compilation?

E' una via di mezzo fra interpretazione e compilazione. Cerca di abbattere i costi di rallentamento dell'interpretazione avendo un codice compilato, pero' non dovendo compilare tutto a priori ma solo a chiamata quando serve in esecuzione.

Domanda 13

L'interprete e il compilatore si possono sempre realizzare?

Il compilatore puo' essere sempre realizzato anche perche' puo' girare su una macchina esterna rispetto a i linguaggi in input-output. Mentre l'interprete ha un costo di implementazione maggiore, quindi non e' detto che sia supportato. Bisogna poi tenere conto anche del livello di espressionalita' dei linguaggi. Un linguaggio con potere maggiore non puo' essere convertito in uno di potere minore.

Domanda 14

Che cosa e l'implementazione via kernel?

Si divide il linguaggio $\mathcal{L}_0$ in due parti:

- *Primitive: operazioni essenziali*
- *Derivate: operazioni che poi vengono ridotti (tradotte) nelle primitive*

Il vantaggio e' che dobbiamo implementare solo poche operazione, ovvero le primitive, il resto e' solo traduzione statica.

Domanda 15

Quando si parla di bootstrapping?

E' quando per implementare il linguaggio $\mathcal{L}$ scriviamo un compilatore o un interprete in $\mathcal{L}$.

1.2 Capitolo 2 – Descrivere un linguaggio di programmazione

Domanda 16

Quali sono i livelli di descrizione di un linguaggio?

1. *Sintassi: parole (token)*
2. *Semantica: costruzione delle frasi (es. assegnazione tipo sbagliato)*
3. *Pragmatica: come lo implementiamo*

Domanda 17

Sintassi: qual e l'aspetto lessicale e quale quello grammaticale di un linguaggio?

Il lessico sono le parole (token) consentite, mentre la grammatica e' l'ordine delle parole.

Domanda 18

Cos'e un alfabeto? Cos'e una parola o stringa? Cos'e A^* ? Tale insieme e enumerabile?

Un alfabeto A e' un insieme finito di simboli non vuoti. Una parola (o stringa) e' una sequenza di lettere dell'alfabeto. Definiamo A^ l'insieme di tutte le parole finite (inclusa la vuota) di A :*

$$A^* = \bigcup_{i \geq 0} A^i$$

Se A e' finito allora possiamo ordinare per lunghezza tutte le parole, ottenendo una sequenza numerabili di numeri naturali.

Domanda 19

Definizione di potenza di una stringa. Definizione di potenza di un linguaggio. Definizione di chiusura/iterazione (o stella di Kleene) di un linguaggio.

La n -esima potenza della stringa w e' definita come:

$$w^n = \begin{cases} \epsilon & n = 0 \\ ww^{n-1} & n > 0 \end{cases}$$

Dato un alfabeto A e L l'insieme di stringhe (linguaggio), definiamo la stella di Kleene:

$$L^* = \bigcup_{i \geq 0} L^i$$

con

- $L^0 = \{\epsilon\}$
- $L^{n+1} = LL^n$

Domanda 20

Definizione di grammatica libera da contesto. Come si deriva una stringa? Qual e il linguaggio generato da una grammatica libera?

Una CFG (PDA) e' una quadra

$$CFG = (V, T, P, S)$$

- V : variabili (non terminali)
- T : terminali
- P : produzioni ($A \rightarrow a|b$)
- S : simboli di start

Data una CFG deriviamo w da v se esiste una transizione riflessiva:

$$v \rightarrow w_i \rightarrow w$$

ovvero

$$\frac{}{v \rightarrow^* w_i} \quad \frac{v \rightarrow^* w}{v \rightarrow^* w_i w_i \rightarrow w}$$

Il linguaggio generato da una CFG è l'insieme di tutte le stringhe di terminali derivabili da S (simboli di start)

Domanda 21

Cos'è un albero di derivazione? Cos'è una derivazione canonica sinistra/destra? Esiste una corrispondenza biunivoca tra alberi di derivazioni e derivazioni canoniche?

Un albero di derivazione e' una rappresentazione grafica di una derivazione in una CFG.

Per calcolare l'albero di derivazione dobbiamo prima espandere i non terminali, quindi dobbiamo decidere se partire da destra o sinistra (sono le derivazioni canoniche). Quindi l'albero di derivazione e' equivalente alle espansioni canoniche (sono biunivoche).

Domanda 22

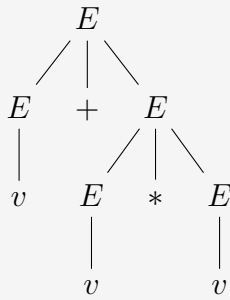
Quando una grammatica è ambigua? (fare un esempio) Quando un linguaggio è ambiguo? (fare un esempio)

Una grammatica e' ambigua quando data una stringa esistono più di un albero di derivazione **distinto**:

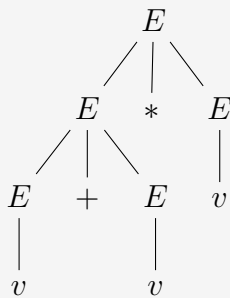
$$E \rightarrow E + E | E * E | (E) | v$$

ESEMPIO: $v+v*v$

1. $(v + v) * v$



2. $v + (v * v)$



Un linguaggio e' ambiguo quando possiamo formularlo con una grammatica ambigua.
ESEMPIO: $L = \text{espressione su } v$.

Domanda 23

E' possibile rimuovere l'ambiguita' dalla grammatica delle espressioni aritmetiche? Come?

Si e' possibile introducendo priorit  fra gli operatori aritmetici.

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid v
 \end{aligned}$$

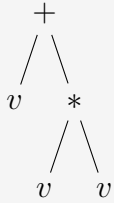
Ovvero essendo che il $*$ sta a un livello piu' profondo del $+$ allora ha una precedenza maggiore.

Domanda 24

Cos'e' l'albero di sintassi astratta? Che differenza c'e' tra sintassi concreta e sintassi astratta? Cos'e' lo zucchero sintattico?

L'AST e' l'albero ripulito dai non terminali e terminali dell'albero di derivazione e anche dalla sintassi non essenziali. E' una rappresentazione logica della struttura essenziali del linguaggio di programmazione.

*ESEMPIO: $v+v*v$*



Lo zucchero sintattico sono tutti quei costrutti di un LP che servono a migliorare la leggibilità del programma rispetto alla implementazione (es. i `for` sono sotto `while`).

Domanda 25

Fare esempi di vincoli sintattici contestuali. Possono essere catturati attraverso grammatiche libere?

I vincoli sintattici contestuali sono regole che non dipendono solo dalla forma della stringa, ma dal contesto in cui compaiono simboli o costrutti. (ovvero tipo uso di variabili non prima dichiarate). Queste non possono essere rilevate da CFG.

Domanda 26

Cosa s'intende per semantica statica? E per semantica dinamica?

La semantica statica copre tutti i controlli statici sul linguaggio (es. conflitto di tipi, var. non dichiarata), mentre la semantica dinamica scrive il comportamento del programma durante l'esecuzione (es. if-else, operazioni aritmetiche).

Domanda 27

Elencare le varie fasi in cui si articola un compilatore. Descrivere in dettaglio ogni singola fase.

1. *Lexer: genera il sorgente in una lista di token*
2. *Parser: Genera AST, verifica regole grammaticali (ordine parole)*
3. *Analisi semantica statica: controllo su conflitti di tipi, dichiarazione di variabili*
4. *Generazione IR*
5. *Ottimizzazione (facoltativa)*
6. *Generazione del codice per la architettura*

Domanda 28

A chi serve definire la semantica dinamica di un linguaggio e perché?

La semantica dinamica è la base teorica dell'esecuzione degli interpreti o del derivato di un compilatore. Serve all'implementatore/sviluppatore.

Domanda 29

Quali tecniche si usano per dare semantica dinamica ad un linguaggio di programmazione?

1. *Semantica operativa: descrive l'esecuzione come una sequenza di passi su una macchina astratta*
2. *Semantica denotazionale: associa a ogni costrutto del linguaggio un oggetto matematico*
3. *Semantica assiomatica: descrive il comportamento tramite logica e predicati (Hoare logic) (tipo $\text{precondizione} \rightarrow \text{comando} \rightarrow \text{postcondizione}$)*

Domanda 30

Imparare le regole di semantica operativa SOS per il semplice linguaggio presentato a lezione. Regole di valutazione: interna vs esterna, sinistra vs destra vs parallela.

La valutazione interna descrive ogni passaggio intermedio, mentre la esterna solo il risultato finale.

La valutazione destra, sinistra servono per indicare la priorità di valutazione in caso di due argomenti. Parallela invece li valuta contemporaneamente.

$$\begin{array}{c}
\langle V, \sigma \rangle \rightarrow_e \langle \underbrace{\sigma(V), \sigma}_{\text{stato terminale}} \rangle \text{ VAR} \\
\\
\frac{\langle e_0, \sigma \rangle \rightarrow_e \langle e'_0, \sigma' \rangle}{\langle e_0 + e_1, \sigma \rangle \rightarrow_e \langle e'_0 + e_1, \sigma' \rangle} \text{ SUM1} \qquad \frac{\langle e_1, \sigma \rangle \rightarrow_e \langle e'_1, \sigma' \rangle}{\langle m + e_1, \sigma \rangle \rightarrow_e \langle m + e'_1, \sigma' \rangle} \text{ SUM2} \\
\\
\frac{}{\langle m + m', \sigma \rangle \rightarrow_e \langle P, \sigma \rangle} \text{ SUM3} \quad \text{se } P = m + m' \\
\\
\frac{\langle e_0, \sigma \rangle \rightarrow_e \langle e'_0, \sigma' \rangle}{\langle e_0 - e_1, \sigma \rangle \rightarrow_e \langle e'_0 - e_1, \sigma' \rangle} \text{ SUB1} \qquad \frac{\langle e_1, \sigma \rangle \rightarrow_e \langle e'_1, \sigma' \rangle}{\langle m - e_1, \sigma \rangle \rightarrow_e \langle m - e'_1, \sigma' \rangle} \text{ SUB2} \\
\\
\frac{}{\langle m - m', \sigma \rangle \rightarrow_e \langle P, \sigma \rangle} \text{ SUB3} \quad \text{se } m \geq m' \text{ e } P = m - m'
\end{array}$$

Domanda 31

Cosa s'intende per **pragmatica**? Cosa si intende per **implementazione** di un linguaggio?

Per pragmatica di un linguaggio si intende l'insieme degli aspetti legati all'uso del linguaggio da parte del programmatore e agli effetti concreti dell'esecuzione dei programmi, cioè come il linguaggio si comporta nella pratica (efficienza, gestione della memoria, effetti collaterali, modello di esecuzione, ecc.).
Con implementazione intendiamo la realizzazione concreta del linguaggio attraverso un interprete o compilatore.

1.3 Capitolo 3 – Analisi lessicale, linguaggi regolari

Domanda 32

Cosa fa l'analizzatore lessicale?

Il lexer prende il sorgente e restituisce una lista di token (identificatori, numeri, operazioni, ecc)

Domanda 33

Cos'è un token?

Un token è un identificatore che il programmatore del compilatore ha definito per specificare che quella parola (stringa) sia di quella tipologia.

Domanda 34

Cos'è un pattern? Come lo si rappresenta? Cos'è un lessema?

Un pattern è una descrizione formale che specifica quali stringhe appartengono a una certa categoria lessicale.

- *Regex*
- *NFA-DFA*

L'output del lexer è una lista di tuple (token, lessema)

Domanda 35

Definizione di espressioni regolari (sintassi) e di linguaggio associato (semantica).

Dato alfabeto A con lettere $a_i \in A$ definiamo espressioni regolari:

$$E \rightarrow \emptyset \mid \epsilon \mid a_i \mid r \cdot r \mid E \text{ or } E \mid E^*$$

Definiamo linguaggio regolare:

- $L[\emptyset] = \emptyset$
- $L[\epsilon] = \{\epsilon\}$
- $L[a_i] = \{a_i\}$, per ogni simbolo $a_i \in A$
- $L[E_1 \cdot E_2] = \{xy \mid x \in L[E_1], y \in L[E_2]\}$ (concatenazione)
- $L[E_1 \mid E_2] = L[E_1] \cup L[E_2]$ (unione / or)
- $L[E^*] = \bigcup_{n \geq 0} L[E]^n$ (stella di Kleene)

Domanda 36

Quali sono i linguaggi regolari? I linguaggi finiti sono tutti regolari? Esistono linguaggi infiniti regolari?

I linguaggi regolari sono una classe dei linguaggi formali. Sono descritti da:

- *Espressioni regolari*
- *Automi a stati finiti (DFA-NFA)*
- *CFG*

I linguaggi finiti sono tutti regolari, infatti essendo finiti possiamo definire una espressione regolare ad-hoc:

$$L = a, bb, ccc \Rightarrow a \mid bb \mid ccc$$

Esistono pero' linguaggi infiniti (solo alcuni non tutti!) che sono regolari:

$$L = \{\epsilon, a, aa, aaa, \dots\} \Rightarrow aA \mid \epsilon$$

Domanda 37

Definizione di equivalenza tra espressioni regolari. Elencare alcune leggi di equivalenza.

Una definizione regolare su alfabeto A è costituita da una lista di definizioni regolari

$$d_1 := E_1$$

$\vdots$

$$d_n := E_n$$

definiamo d_i simboli nuovi da utilizzare per ampliare l'alfabeto $(A \cup d_i)$.

ESEMPIO:

$$\text{identifier} := \text{letter}(\text{letter}|\text{digit})^*$$

$$\text{letter} := [a - zA - Z]$$

$$\text{digit} := [0 - 9]$$

Dato E_1 e E_2 come espressioni regolari, definiamo equivalenti L_1 e L_2 se

$$L_1[E_1] = L_2[E_2]$$

Domanda 38

Cos'è una definizione regolare e a cosa serve?

E' una lista di espressioni regolari, e servono per ampliare l'alfabeto di un linguaggio attraverso lettere + espressioni.

Domanda 39

Definizione di NFA (automa finito nondeterministico). Discutere cosa si intende per nondeterminismo. Mettere in relazione la definizione formale con la rappresentazione grafica come diagramma di transizioni.

Un NFA è una quintupla $\{\Sigma, Q, \delta, q_0, F\}$ dove:

- Σ : input
- Q : stati totali
- δ : funzione di transizione

- $q_0 \subseteq Q$: stati iniziali
- $F \subseteq Q$: stati finali

Con nondeterminismo si intende quando esiste almeno uno stato q , che ha input a t.c.

$$\delta(q, a) = q'$$

$$\delta(q, a) = q''$$

Ovvero se esiste almeno uno stato che puo' prendere due alternative possibili ($|\delta(q, a)| \geq 1$).

Possiamo disegnare un NFA come un grafo nel quale:

- Lo stato iniziale e' q_0
- Ogni stato $q \in Q$ corrisponde a un nodo (cerchio) del grafo.
- Gli stati finali F sono rappresentati con doppio cerchio.
- Ogni transizione $q' \in \delta(q, a)$ è rappresentata da un arco (freccia) orientato da q a q' etichettato con il simbolo a .

Domanda 40

Come si definisce il linguaggio accettato da un NFA? Definizione di equivalenza tra NFA.

Il linguaggio accettato per stato finali di un NFA si definisce come l'insieme generato dallo stato q_0 + i valori consumati dalle transizioni δ fino ad arrivare a stati finali in F .

Due NFA sono equivalenti quando i loro linguaggi accettati e rifiutati combaciano.

Domanda 41

Definizione di DFA (automa finito deterministico). Discutere cosa si intende per determinismo. Mostrare che un DFA è un caso speciale di NFA, ovvero la classe dei DFA è un sottoinsieme della classe degli NFA.

Un DFA è una quintupla $\{\Sigma, Q, \delta, q_0, F\}$ dove:

- Σ : input
- Q : stati totali
- δ : funzione di transizione
- $q_0 \subseteq Q$: stati iniziali
- $F \subseteq Q$: stati finali

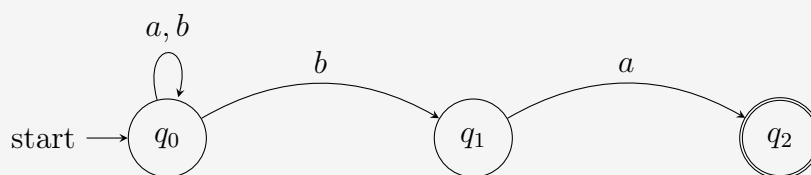
Ma la cardinalità di tutte le transizioni è 1, $|\delta(q, a)| = 1$.

Per determinismo definiamo che ad ogni stato non possiamo scegliere fra alternative ma abbiamo una strada singola.

Domanda 42

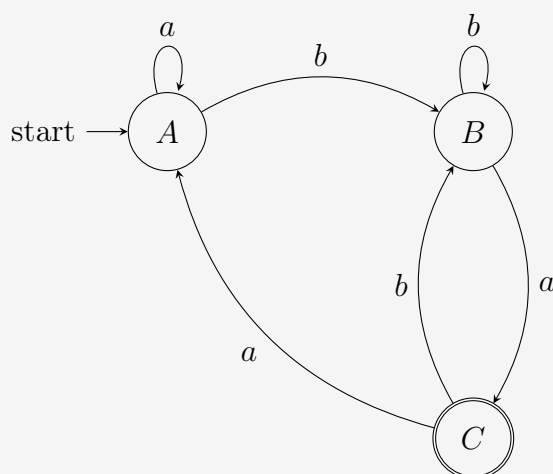
Dato un NFA, come si ricava un equivalente DFA? Descrivere la costruzione per sottoinsiemi. Definizione di epsilon-closure e algoritmo associato. Qual e la complessita della costruzione per sottoinsiemi nel caso pessimo?

Per dimostrare che $DFA \subset NFA$, dobbiamo utilizzare la ϵ -closure.



Ora calcoliamo i nuovi stati grazie alla funzione di transizione Δ del DFA:

- $A = \epsilon\text{-closure}(0): \{0\}$
- $\Delta(A, a) = \{\epsilon\text{-closure}(\underbrace{\text{mossa}(A, a)}_{\text{esclusa partenza}})\} = \epsilon\text{-closure}(\{0\}) = \{0\}$
- $\Delta(A, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$
- $\Delta(B, a) = \epsilon\text{-closure}(\{0, 2\}) = \{0, 2\} = C$
- $\Delta(B, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$
- $\Delta(C, a) = \epsilon\text{-closure}(\{0\}) = \{0\} = A$
- $\Delta(C, b) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\} = B$



DFA ottenuto per costruzione per sottoinsiemi.

La ϵ -closure di q e' l'insieme degli stati raggiungibile da q solo con mosse ϵ .
Se con un NFA ha cardinalita n allora il suo DFA nel caso peggiore ha 2^n di cardinalita'.

Domanda 43

Enunciare il teorema che dice che la classe dei linguaggi riconosciuti da NFA coincide con la classe dei linguaggi riconosciuti da DFA.

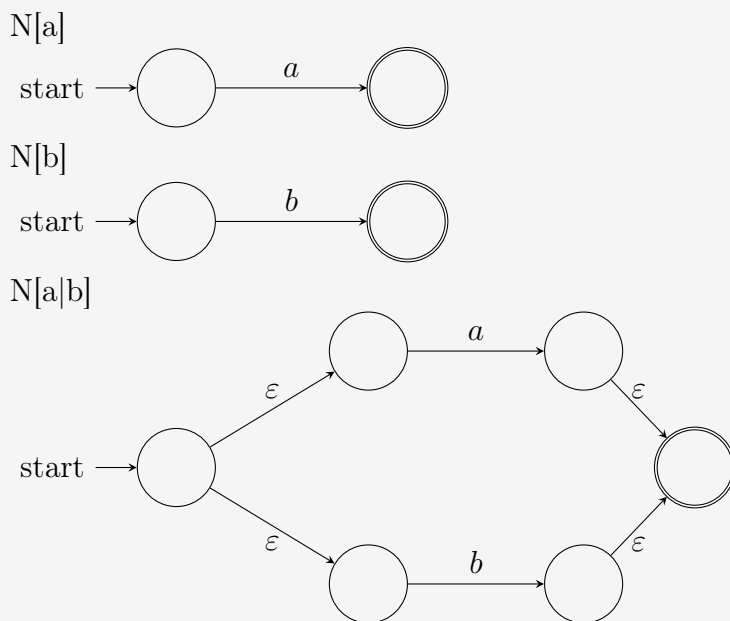
Teorema: Sia $N = (\Sigma, Q, \delta, q_0, F)$ un NFA e sia M_N l'automa ottenuto per sottoinsiemi. Allora M_N e' un DFA e si ha che:

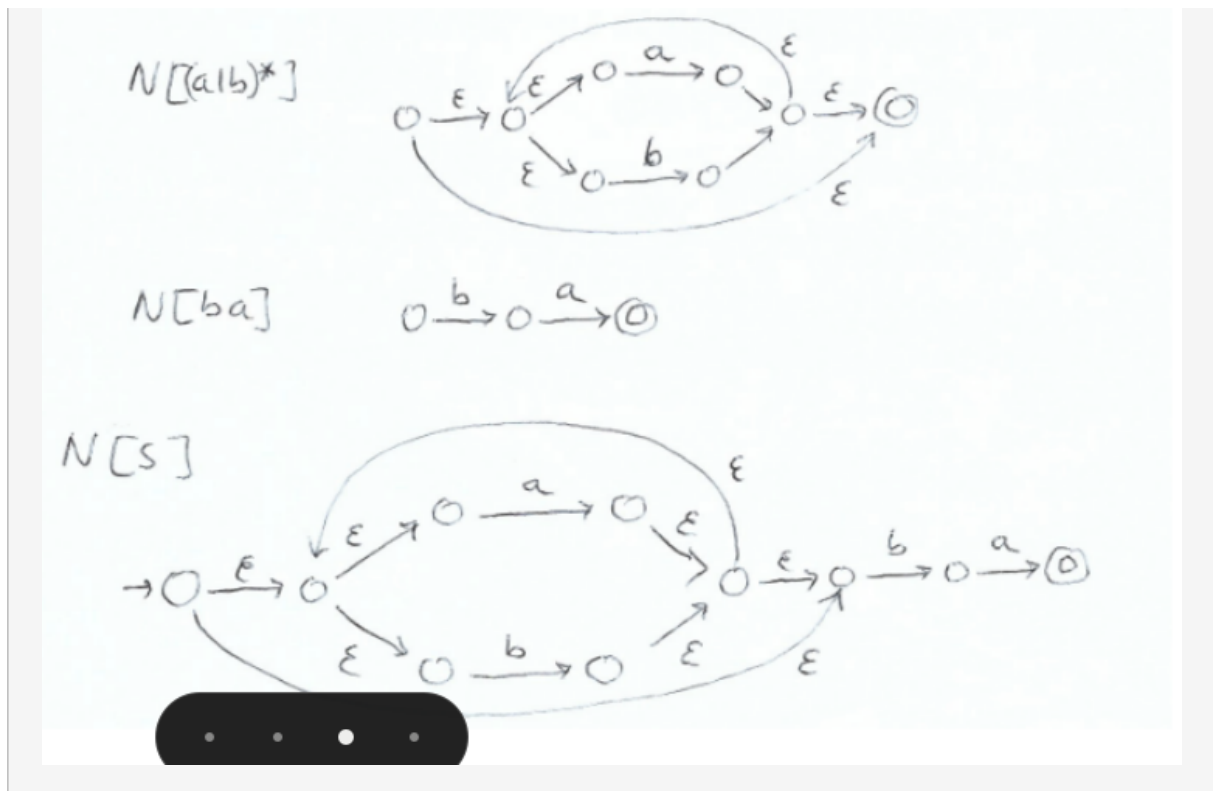
$$L[N] = L[M_N] \quad (1)$$

Domanda 44

Come si costruisce un NFA a partire da una espressione regolare, in modo tale che il linguaggio riconosciuto dall'NFA sia lo stesso del linguaggio associato all'espressione regolare?

Dato $S = (a|b)^*ba$
Partiamo dalla radice (bottom-up):





Domanda 45

Definizione di grammatica regolare.

La dice grammatica G e' regolare sse e' generata da un linguaggio regolare.
ESEMPIO:

$$G := \begin{cases} A \rightarrow aB \mid \epsilon \\ B \rightarrow bA \mid \epsilon \end{cases}$$

$$G := A \rightarrow abA \mid a \mid \epsilon$$

$$G := (ab)^*(a \mid \epsilon)$$

Domanda 46

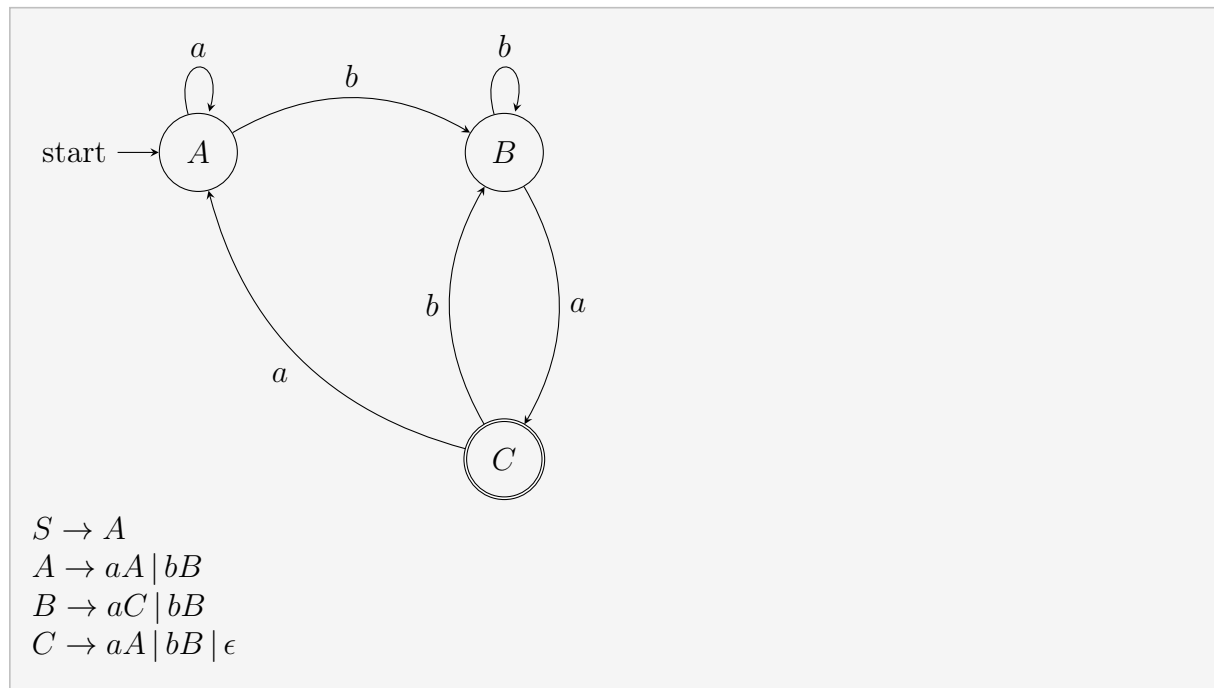
Come si associa ad una grammatica regolare un equivalente NFA?

$$NFA \Rightarrow DFA \Rightarrow \text{Grammatica regolare}$$

Domanda 47

Dato un DFA, come si costruisce una grammatica regolare equivalente?

I nodi saranno i terminali, mentre gli archi i non terminali.



Domanda 48

Data una grammatica regolare, come si costruisce un'espressione regolare equivalente?

Semplifichiamo la grammatica rimuovendo i simboli inutili

$A \rightarrow aA \mid \epsilon \mid bc$

$C \rightarrow bC \mid \epsilon \mid aD$

$D \rightarrow aD \mid bD$ (questa non finisce mai)

diventa

$A \rightarrow aA \mid \epsilon \mid bC$

$C \rightarrow bC \mid \epsilon$

Ora ricaviamo l'espressione regolare associata:

$C = b^*$

$A = a^*b^*$

Domanda 49

Descrivere, con un diagramma riassuntivo, tutte le relazioni fra i formalismi introdotti: NFA, DFA, grammatiche regolari, espressioni regolari.

$$\begin{array}{ccc}
\text{Grammatiche regolari} & \Rightarrow & \text{NFA} \\
\Uparrow & & \Downarrow \\
\text{Espressioni regolari} & \Leftarrow & \text{DFA}
\end{array}$$

Domanda 50

Definizione di stati equivalenti in un DFA. Quando due stati di un DFA sono distinguibili?

Due stati sono equivalenti quando per ogni stringa $w \in \{\Sigma + \epsilon\}$:

$$\begin{aligned}
\delta^*(q_0, w) \in F & \iff \delta^*(q_1, w) \in F \\
L[\text{DFA}, q_0] &= L[\text{DFA}, q_1]
\end{aligned}$$

Nota che δ^ e' la funzione di transizione completa su tutta la stringa.*

Due stati sono distinguibili quando:

$$\delta^*(q_0, w) \in F \text{ e } \delta^*(q_1, w) \notin F$$

Domanda 51

Come funziona l'algoritmo iterativo con tabella a scala per produrre le classi di equivalenza di stati di un DFA?

Dato un DFA con $q_0, \dots, q_i$ stati e q_i stato terminale, costruiamo la tabella a scala:

- 1. Le x sara' numerato da 0 a i - 1*
- 2. Le y sara' numerato da i a 1*
- 3. Flag su l'incrocio fra tutti gli elementi con gli stati terminali*
- 4. Controlliamo per ogni cella , ip. $q_k q_m$, ogni movimento per ogni lettera, se otteniamo anche solo una nuova coppia che e' flaggata allora flaggiamo anche qua*
- 5. Se abbiamo delle coppie di stati non flaggate allora sono equivalenti*
- 6. Costruiamo un nuovo grafo unendo le coppie equivalenti*
- 7. Ricontrolliamo*

Domanda 52

Una volta determinate le classi di equivalenza degli stati di un DFA, come si costruisce l'automa minimo associato?

1. Date le coppie di stati di equivalenza
2. Costruiamo un nuovo grafo unendo le coppie identica fra le loro relazioni
3. Ricostruiamo un nuovo DFA
4. Controlliamo se questo sia minimo

Ora abbiamo un DFA minimo!

Domanda 53

Cos'è Lex? Qual è il suo input e il suo output?

Lex è un generatore di Lexer:

- Input: file `.l` nel quale specifichiamo le definizioni regolari
- Output: file `.c` che realizza l'automa riconoscitore

Domanda 54

Qual è la struttura di un file `.l`? Come funziona l'analizzatore lessicale prodotto da Lex?

E' diviso in 3 parti:

1. Dichiarazioni (definizioni regolari)
2. Regole (pattern $\Rightarrow$ C code)
3. Funzioni ausiliarie

L'automa in C implementa il DFA riconoscitore delle espressioni regolari e restituisce `<TOKEN, lessema>`.

Domanda 55

Come si interfaccia Lex con Yacc?

Yacc chiama Lex attraverso una funzione `yylex()`, lex ritorna la tuple `<TOKEN, lessema>` e YACC è un generatore di parser (analizzatore sintattico).

Domanda 56

Intestazione e dimostrazione del pumping lemma.

Se L e' un linguaggio regolare allora $\exists N \geq 1$ t.c. per ogni stringa accettata in L ($uvw \in L$) e' $|uvw| \geq N$:

- $|uv| \leq N$
- $v \geq 1$
- $\forall k \geq 0 \quad uv^k w \in L$

DIMOSTRAZIONE: Sia $DFA = (\Sigma, Q, \delta, q_0, F)$ che genera L :

$$|DFA| = N \quad (2)$$

Consideriamo una stringa $|uvw| \geq N$, durante la lettura delle prime N lettere siamo passati da $N+1$ stati (anche iniziale), ed essendo un DFA per definizione abbiamo solo N stati. Quindi in uno stato ci siamo passati piu' volte.

Chiamiamo q_i lo stato passato piu' volte, definiamo che

- x e' la parte di string fino a q_i
- v e' la parte di loop
- w e' la parte post q_i

Ovvero che allo stato q_i se leggiamo certi caratteri (quelli dentro v) otteniamo

$$\delta^*(q_i, v) = q_i$$

la lunghezza di v non importa tanto e' un loop che consuma tutti i caratteri compatibili con il loop, ovvero

$$uv^k w \in L \quad \forall k \geq 0$$

Domanda 57

Come si puo utilizzare il pumping lemma (a rovescio) per dimostrare che un linguaggio non e regolare?

Dimostriamo che $L = \{a^n b^n | n \geq 1\}$ non sia regolare.

Assumiamo per assurdo che L sia regolare, allora vale il pumping lemma che ci dice:

- Fissata una lunghezza $N \geq 1$
- Data la stringa $uvw \in L$ e $|uvw| \geq N$:
 1. $|uv| \leq N$
 2. $|v| \geq 1$
 3. $\forall k \geq 0 \quad uv^k w \in L$

La nostra stringa e' $a^N b^N$, quindi $uv = a^N$ e $w = b^N$.

Scomponiamo $uv = a^N$ in $u = a^m$ e $v = a^{N-m}$ ($m < N$), pero' per n.3 sappiamo

che

$$\begin{aligned} uv^k w &\in L \\ \Rightarrow a^m a^{k(N-m)} b^N &\in L \quad \forall m < N \quad \forall k \geq 0 \\ \Rightarrow a^m a^{kN-km} b^N &\end{aligned}$$

Ipotizziamo $N = 2, k = 2$:

$$\Rightarrow a^m a^{4-2m} b^2$$

Ipotizziamo $m = 1$

$$\begin{aligned} \Rightarrow a^1 a^2 b^2 \\ \Rightarrow aaabb \notin L \end{aligned}$$

$\rightarrow$ assurdo! Quindi L non e' regolare.

Domanda 58

Quali sono le proprieta di chiusura dei linguaggi regolari? Quali proprieta si possono decidere (ovvero verificare alitmicamente)?

La classe regolare e' chiusa per tutto, ovvero unione, intersezione, concatenazione, Kleene, complemento.

1.4 Capitolo 4 – Analisi sintattica, linguaggi liberi

Domanda 59

Cosa si intende per analisi sintattica? Cos'e un parser?

L'analisi sintattica e' la fase del compilatore dove data una lista di token si controlla che rispetti la grammatica del linguaggio (ordine dei token). Il parser e' il componente che esegue l'analisi sintattica e restituisce AST.

Domanda 60

Definizione di automa a pila nondeterministico (PDA). Definizione di configurazione (o descrizione istantanea), mossa in un passo e mossa in piu passi.

Un PDA (automa a pila non-deterministico) e' una 7-pla $(\Sigma, Q, \Gamma, \delta, q_0, \perp, F)$ dove:

- Σ : input
- Q : stati totali

- Γ : insieme simboli pila
- δ : funzione di transizione
- q_0 : stato iniziale
- $\perp \in \Gamma$: simbolo iniziale sulla pila
- F : stato finale

Domanda 61

Definizione di linguaggio accettato da un PDA per stato finale o per pila vuota. Sono equivalenti queste due modalita di riconoscimento?

Due modalita' di riconoscimento:

- *Stato finale:*

$$\{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q_f, \epsilon, \gamma \quad q_f \in F)\}$$

- *Pila vuota:*

$$\{w \in \Sigma^* | (q_0, w, \perp) \vdash_N^* (q, \epsilon, \epsilon)\}$$

- q_0 stato iniziale
- w stringa ancora non letta
- $\perp$ pila vuota
- q_f stato finale
- ϵ lettera terminale
- γ contenuto pila (completa)

Le due modalita' sono equivalenti per un teorema.

Domanda 62

Mostrare come data una grammatica libera G sia possibile costruire un PDA P equivalente. E possibile costruire, dato un PDA P , una grammatica equivalente G ? Concludere che la classe dei linguaggi liberi coincide con la classe dei linguaggi riconosciuti da PDA.

Domanda 63

Quali sono le proprieta di chiusura dei linguaggi liberi? L'intersezione di un linguaggio libero con un regolare e un linguaggio libero?

Domanda 64

Intestazione e dimostrazione del Pumping Theorem per i linguaggi liberi.

Domanda 65

Come si può utilizzare tale teorema (a rovescio) per dimostrare che un linguaggio non è libero?

Domanda 66

Classificazione di Chomsky delle grammatiche e dei linguaggi. Definizione di grammatica dipendente dal contesto e di grammatica monotona. Quale tipo di automi corrisponde ad ogni classe?

Domanda 67

Definizione di DPDA (automa a pila deterministico) e di linguaggio libero deterministico.

Domanda 68

La classe dei linguaggi liberi deterministici è strettamente inclusa in quella dei linguaggi liberi? Contiene strettamente la classe dei linguaggi regolari?

Domanda 69

Che cosa dice la prefix property e perché è interessante per i DPDA?

Domanda 70

Usando un endmarker \$, si può riconoscere un linguaggio libero deterministico che non gode della prefix property anche per pila vuota? Come?

Domanda 71

Un linguaggio libero deterministico è ambiguo?

Domanda 72

Proprietà di chiusura dei linguaggi liberi deterministici: chiusi per complementazione, ma non per intersezione né per unione.

Domanda 73

Da cosa si parte per costruire un analizzatore sintattico (ovvero parser)? Da una espressione regolare? Da una grammatica libera? Da un PDA?

Domanda 74

Cosa prende in input e cosa produce in output un parser?

Domanda 75

Che differenza c'è tra un parser nondeterministico ed uno deterministico?

Domanda 76

Quali sono le due tecniche essenziali per costruire parsers?

Domanda 77

Le tecniche top-down e bottom-up in che cosa differiscono?

Domanda 78

Quali tipi di grammatiche non sono adatte al top-down parsing? Quali tipi di produzioni sono poco adatte al bottom-up parsing?

Domanda 79

Cosa sono le produzioni epsilon e cosa sono i simboli non-terminali annullabili?

Domanda 80

Come si può trasformare una grammatica G che contiene produzioni epsilon in una grammatica G' che non ne contiene, preservando il linguaggio a meno di epsilon?

Domanda 81

Cosa sono le produzioni unitarie e cosa sono le coppie unitarie?

Domanda 82

Come si può trasformare una grammatica G che contiene produzioni unitarie in una equivalente G' che non ne contiene?

Domanda 83

Data una grammatica G , quali sono i suoi simboli utili? Quali sono i suoi simboli generatori? Quali i suoi simboli raggiungibili?

Domanda 84

Come si calcolano i generatori? Come si calcolano i raggiungibili?

Domanda 85

In che modo si eliminano i simboli inutili di una grammatica? E importante l'ordine delle operazioni da svolgere?

Domanda 86

Quando si dice che una grammatica è ricorsiva sinistra? Come si elimina la ricorsione sinistra immediata? E quella non immediata? Perché serve eliminare la ricorsione sinistra?

Domanda 87

Cosa vuol dire fattorizzare a sinistra una grammatica? Perché serve fattorizzare?

Domanda 88

Cos'è un parser a discesa ricorsiva?

Domanda 89

Definizione di $\text{First}(a)$ e di $\text{Follow}(A)$.

Domanda 90

Algoritmi per calcolare $\text{First}(a)$ e $\text{Follow}(A)$.

Domanda 91

Come e fatta e come si riempie una tabella di parsing LL(1)?

Domanda 92

Quando una grammatica G si dice di classe LL(1)? Quali sono le condizioni necessarie e sufficienti per G affinché sia di classe LL(1)?

Domanda 93

Perché un parser di questo tipo è chiamato LL?

Domanda 94

Come funziona il parser LL(1) con una pila?

Domanda 95

È vero che ogni linguaggio regolare è pure LL(1)?

Domanda 96

Come sono definiti $\text{First}_k(a)$ e $\text{Follow}_k(A)$ per $k \geq 2$?

Domanda 97

Come si definisce una grammatica LL(k)? Ed un linguaggio LL(k)? Come si relazionano tra di loro?

Domanda 98

Che relazione esiste tra grammatiche LL(k) e grammatiche ambigue? E con le grammatiche ricorsive sinistre?

Domanda 99

Esistono linguaggi liberi che non sono LL(k) per nessun k? Ed esistono linguaggi liberi deterministici che non sono LL(k) per nessun k?

Domanda 100

Cos'è un parser bottom-up (o shift-reduce)? Qual è il suo input e il suo output? Perché sono chiamati parser LR?

Domanda 101

Che tipo di conflitti si possono presentare in un parser del genere? Quando si presenta un conflitto (shift/reduce o reduce/reduce), quale azione bisogna scegliere?

Domanda 102

Cos'è un prefisso viabile? Come lo si definisce in termini di una grammatica?

Domanda 103

Cos'è un item LR(0)? Come si generano tutti gli item di una grammatica (aumentata con un simbolo iniziale nuovo S')?

Domanda 104

Come è fatto il NFA dei prefissi viabili? Come si ricava il DFA dei prefissi viabili, detto anche automa canonico LR(0)?

Domanda 105

Come è fatta una tabella di parsing LR(0)? Come la si riempie a partire dall'automato canonico LR(0)? Quando una grammatica è di classe LR(0)?

Domanda 106

Come è fatto il parser LR(0) che utilizza la tabella di parsing LR(0)? Quanti stack servono?

Domanda 107

Esistono grammatiche non LR(0)? Fare un esempio semplice.

Domanda 108

Come è fatta e come si riempie una tabella di parsing SLR(1)? Cosa vuol dire l'acronimo SLR? Perché si mette 1 come parametro?

Domanda 109

Quando una grammatica è di classe SLR(1)? Esistono grammatiche non di classe SLR(1)?

Domanda 110

Cos'è un item LR(1)? Come si costruisce il NFA LR(1)? E come l'automa canonico LR(1)?

Domanda 111

Come è fatta e come si riempie una tabella di parsing LR(1)?

Domanda 112

Come è fatta una tabella di parsing LALR(1)? Quando una grammatica è di classe LALR(1)?

Domanda 113

Esistono grammatiche LALR(1) che non sono SLR(1)? Esistono grammatiche LR(1) che non sono LALR(1)? Mostrare una grammatica che è LR(1) ma non LALR(1).

Domanda 114

Come si può generalizzare l'idea per ogni $k \geq 2$? Ovvero quando una grammatica è di classe LR(k), SLR(k) o LALR(k)?

Domanda 115

Come si relazionano le grammatiche della famiglia LR (LR, SLR, LALR) al variare di k ? Come si relazionano le grammatiche LR(k) e LL(k) al variare di k ? Le grammatiche LR(k) e LL(k) sono sempre non ambigue? Esistono grammatiche che non sono LR(k) per nessun k ?

Domanda 116

Come si relazionano i linguaggi della famiglia LR(k) rispetto a quelli LL(k)? Esistono linguaggi liberi deterministici che non sono LR(k) per qualche k ? Esistono linguaggi liberi deterministici che non sono LL(k) per qualche k ?

Domanda 117

Esiste un linguaggio regolare che non è LR(0)? Come si relazionano i linguaggi LR(0) rispetto a quelli LL(1)? La prefix property è una condizione necessaria e sufficiente affinché un linguaggio sia di classe LR(0)?

Domanda 118

La classe dei linguaggi SLR(1) coincide con la classe dei linguaggi liberi deterministici?

Domanda 119

Cos'è YACC? Qual è il suo input e il suo output? Come si ottiene un parser eseguibile a partire da un file .y? Come agisce il parser generato da YACC in sintonia con lo scanner generato da Lex?

Domanda 120

Come e la struttura di un file .y di YACC? Cosa sono le regole? Cos'è l'azione semantica?

Domanda 121

È possibile gestire grammatiche ambigue con YACC, specificando le associatività e le priorità fra gli operatori per risolvere l'ambiguità? Come si comporta YACC in presenza di conflitti?

1.5 Capitolo 5 – Fondamenti

Domanda 122

È possibile costruire un programma Check che, preso in input un qualunque programma P, restituisce 1 se P è corretto e 0 se P è scorretto? Ovvero esiste un qualche compilatore che può scovare tutti i possibili errori di un programma?

Domanda 123

Cosa dice il problema della fermata (Halting Problem)? Come si dimostra che il problema non può essere risolto?

Domanda 124

Quando un problema è decidibile?

Domanda 125

Quali sono tipici esempi di proprietà indecidibili per i linguaggi di programmazione?

Domanda 126

Cos'è una Macchina di Turing (MdT)? Che cosa calcola?

Domanda 127

Quando un linguaggio è detto Turing-completo? Cosa afferma la tesi di Church-Turing e perché non si può dimostrare?

Domanda 128

I normali linguaggi di programmazione sono Turing-completi? Cosa afferma il teorema di Jacopini-Bohm?

Domanda 129

Quale relazione esiste tra espressività di un formalismo e decidibilità di proprietà dello stesso? Fare una panoramica prendendo in esame i tre formalismi MdT, PDA, DFA, e le due proprietà $w \in L(M)$ e $L(M_1) = L(M_2)$.